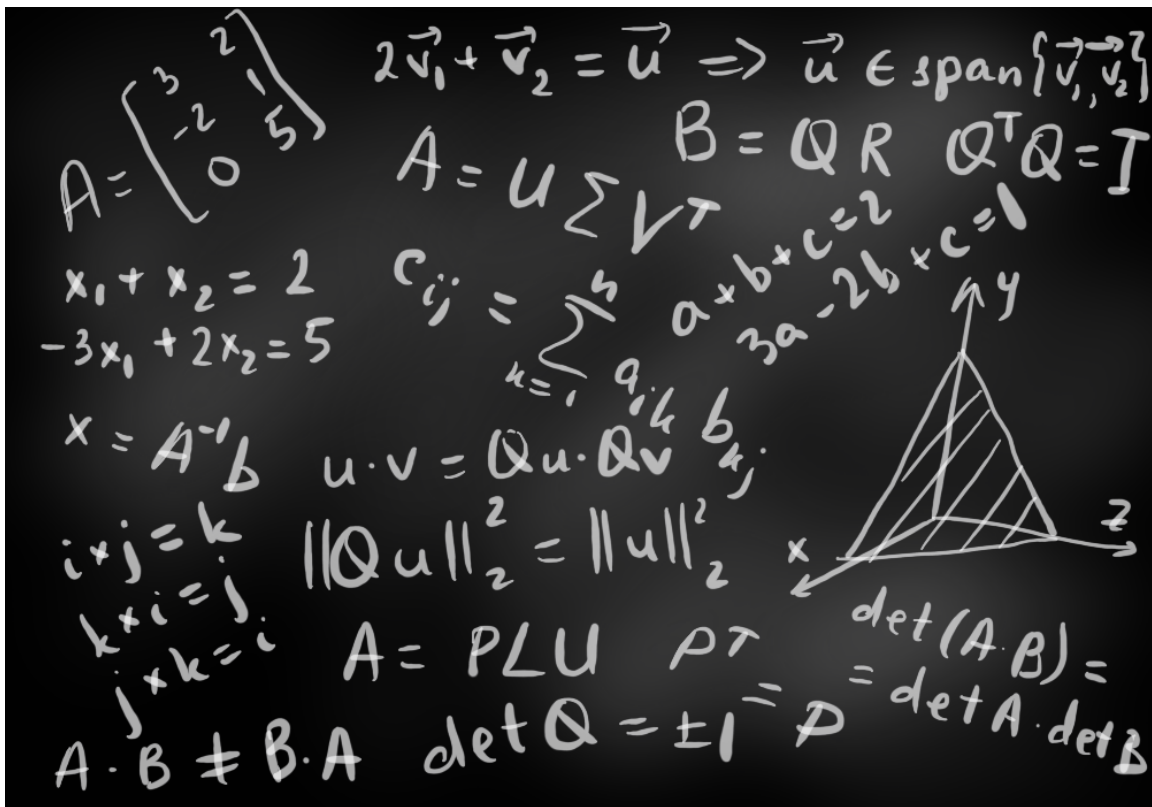
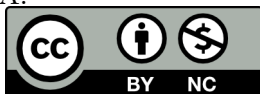


APPLIED PROJECTS FOR AN INTRODUCTORY LINEAR ALGEBRA CLASS

ANNA ZEMLYANOVA



This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.



Contents

About this book	4
A brief introduction to Matlab	5
Advice on good programming style.....	11
Project overview	12
Project 1: Basic operations with matrices in Matlab.....	13
Project 2: Matrix operations and image manipulation	18
Project 3: Matrix multiplication, inversion, and photo filters	24
Project 4: Solving linear systems in Matlab	29
Project 5: Systems of linear equations and college football team ranking (with an example of the Big 12).....	34
Project 6: Convolution, inner product, and image processing revisited	40
Project 7: Norms, angles, and your movie choices	44
Project 8: Interpolation, extrapolation, and climate change.....	49
Project 9: Orthogonal matrices and 3D graphics.....	58
Project 10: Discrete dynamical systems, linear transformations of the plane, and the Chaos Game	64
Project 11: Projections, eigenvectors, Principal Component Analysis, and face recognition algorithms	70
Project 12: Matrix eigenvalues and the Google's PageRank algorithm.....	74
Project 13: Social networks, clustering, and eigenvalue problems.....	79
Project 14: Singular Value Decomposition and image compression	85
Appendices	91
References	107

About this book

This book is based on the Applied Matrix Theory curriculum which the author taught at Kansas State University. Applied Matrix Theory is an introductory linear algebra undergraduate class geared primarily towards engineering students. Unlike a proof-based class that is typically offered to mathematics majors, the focus here is to introduce real life applications at the same time as teaching the standard topics of linear algebra such as matrix operations, linear systems, inverses, vector spaces, determinants, and eigenproblems. While teaching the class, the author of this book struggled with finding realistic applications of the theoretical material which can be reasonably completed during a class period. The search for real life applications has led the author to write this book which hopefully fills the gap between linear algebra theory and applications. Most of the projects in this book can be done with minimal knowledge of programming. It is author's hope that the projects will encourage the exploration of data using simple concepts from linear algebra. The projects are written in Matlab in order to harness the abundance of built in functions and computational power of Matlab. The author attempted to incorporate real data from various open sources into these projects. There are two labs which are dedicated to exploring operations with matrices and to solving linear systems in Matlab (the projects 1 and 4). There are also two labs which use more advanced material typically not covered in an introductory linear algebra class, such as singular value decomposition and normal equations (the projects 8 and 14).

The code in this book has been developed using the 2015a version of Matlab and tested on Matlab 2019a. Depending on the version of Matlab on your computer the syntax and the output might differ slightly. The projects in this book can be adapted to use with GNU Octave as well. The supplementary files necessary to complete some of the labs are available from the author's webpage at <http://www.math.ksu.edu/~azem>.

A variety of printed and Internet sources inspired this book. These sources are acknowledged in the Reference section as well as the sources of data for the projects. This book has been developed under the Open/Alternative Textbook Initiative grant from Kansas State University, the generous support of which is gratefully acknowledged here. The author is immensely grateful to her colleagues, Nathan Albin and Pietro Poggi-Corradini, for proofreading the book and for the corrections. The author is grateful to Jeffrey Sheldon for proofreading the book and suggesting style corrections.

Project 11: Projections, eigenvectors, Principal Component Analysis, and face recognition algorithms

Goals: To use eigenvalues and eigenvectors of a matrix to create a simple face recognition algorithm.

To get started:

- Download the file `lab11.m` and put it in your working directory¹³.
- Download the archive file `database.zip` and unpack it into your working directory. This will produce a folder “database”, which contains 33 image files in the format .pgm¹⁴.

Matlab commands used: `imread`, `size`, `reshape`, `num2str`, `mean`, `uint8`, `double`, `ones`, `eig`, `diag`

What you have to submit: The file `lab11.m` which you will modify during the lab session.

INTRODUCTION

In this project we will apply principal component analysis (PCA) to create a simple face recognition algorithm. The PCA algorithm works by identifying patterns in the data which allows one to reduce the dimension of the dataset. We will use the following steps to apply the PCA algorithm to face recognition:

- Download the database of N images (in our case, $N = 30$). All images must have the same dimensions $m \times n$.
- Input the images `person1.pgm-person30.pgm` into Matlab as arrays. Reshape each of the arrays as a column vector $\mathbf{X}_i$, $i = 1, \dots, N$.
- Compute the mean of all images $\mathbf{X}_{mean} = \frac{1}{N} \sum_{i=1}^N \mathbf{X}_i$.
- Subtract the mean image from each of the vectors $\mathbf{X}_i$: $\tilde{\mathbf{X}}_i = \mathbf{X}_i - \mathbf{X}_{mean}$ and create a matrix $\mathbf{P}$, the i th column of which is equal to the vector $\tilde{\mathbf{X}}_i$. The matrix $\mathbf{P}$ has 30 columns and mn rows where m, n are the dimensions of the images.
- We are interested in the eigenvalues and eigenvectors of the matrix $\mathbf{P}\mathbf{P}^T$. However, the matrix $\mathbf{P}\mathbf{P}^T$ is usually very large. For instance, if our database consists of the images with dimensions $m \times n = 100 \times 100$, then the matrix $\mathbf{P}\mathbf{P}^T$ has the dimensions 10000×10000 , and computation of eigenvalues and eigenvectors becomes unfeasible. Instead, we will use the fact that only at most $N - 1$ eigenvalues of the matrix $\mathbf{P}\mathbf{P}^T$ are non-zero (why?), and those non-zero eigenvalues are the same as non-zero eigenvalues of the matrix $\mathbf{P}^T\mathbf{P}$ (why?). The matrix $\mathbf{P}^T\mathbf{P}$ has the dimensions $N \times N$ which are usually much smaller than the dimensions of the matrix $\mathbf{P}\mathbf{P}^T$. For instance, if there are 30 images in the database (like in our case), then the dimensions of $\mathbf{P}^T\mathbf{P}$ are only 30×30 . Thus, it is much more efficient to solve the eigenvalue problem for the matrix $\mathbf{P}^T\mathbf{P}$.

¹³The printout of the file `lab11.m` can be found in appendices of this book.

¹⁴These files represent a subset of the database of faces from AT&T Laboratories Cambridge www.cl.cam.ac.uk/research/dtg/attarchive/facedatabase.html.

- If $\mathbf{u}_i$ is an eigenvector of the matrix $\mathbf{P}^T\mathbf{P}$ corresponding to a non-zero eigenvalue λ_i , then $\mathbf{v}_i = \mathbf{P}\mathbf{u}_i$ is the eigenvector of the matrix $\mathbf{P}\mathbf{P}^T$ corresponding to λ_i . The eigenvectors of the matrix $\mathbf{P}\mathbf{P}^T$ constitute the set of components of the data with the largest variance between them (which are called principal components).
- Compute the projections of your sample data onto the linear subspace generated by the vectors $\mathbf{v}_i$. The power of the algorithm is in the dimension reduction. Our initial images of faces “live” in the mn -dimensional real space. For instance, if the images are 100×100 pixels, then the dimension of the space is equal to $100^2 = 10000$. If we would try to compare these images to each other pixel by pixel, we would need to compare 10000 pixels. However, not all of these pixels are significant. Principal Component Analysis allows us to identify the important information about the image and compare only that information. For instance, if we are working with a database of 30 images, then we will need to compare only 29 numbers. That is a dimensional reduction from 10000 to only 29! The PCA algorithm is a work horse behind many modern techniques in data mining, including face recognition algorithms.

TASKS

1. Open the file lab11.m and run the first code cell titled %% Input database files into the Matlab:

```
%%Input database files into Matlab
clear;
Database_Size=30;
%Reading images from the database. The image files should be located in the
%subfolder ‘‘database’’
for j=1:Database_Size
    image_read=imread(['database\person' num2str(j) '.pgm']);
    [m,n]=size(image_read);
    P(:,j)=reshape(image_read,m*n,1);
end;
```

This code will input the database images into Matlab, reshape each image from the rectangular array to a column vector, and create the matrix P. Observe the use of the string function `num2str` in the `imread` command and the use of the `reshape` function to change the dimensions of the arrays containing the image data. Instead of the rectangular arrays with the elements corresponding to the pixel colors we now have a set of long vectors where columns of each image have been “stacked” on top of each other.

Variables: Database_Size, m, n, P

2. Compute the mean image by applying the `mean` function to the the matrix P row-wise. Display the mean image by using the function `imshow`.

```
%% Computing and displaying the mean face
mean_face=mean(P,2);
imshow(uint8(reshape(mean_face,m,n)))
```

Variables: mean_face

3. Subtract the mean of the images from each of the columns of the matrix P:

```

%% Subtract the mean face
P=double(P);
P=P-mean_face*ones(1,Database_Size);

```

Notice the conversion to the double type of the array P.

4. Compute the covariance matrix $\mathbf{P}^T\mathbf{P}$ and compute the eigenvalues and the eigenvectors of this matrix $\mathbf{P}^T\mathbf{P}$ by using the built in function `eig`. The function `eig` returns two arrays, one of which has eigenvalues on the main diagonal, and the other has eigenvectors as columns. Observe that one of the eigenvalues is very close to zero.

```

%% Compute the covariance matrix of the set and its eigenvalues and
eigenvectors
[Vectors,Values]=eig(P'*P);
EigenVectors=P*Vectors;

```

Variables: `Vectors`, `Values`, `EigenVectors`

Q1: Why does the matrix $\mathbf{P}^T\mathbf{P}$ have a zero eigenvalue?

5. Display the set of eigenfaces on the same figure by running the code cell %% Display the set of eigenfaces:

```

%% Display the set of eigenfaces
for j=2:Database_Size;
    if j==2
        EigenFaces=reshape(EigenVectors(:,j)+mean_face,m,n);
    else
        EigenFaces=[EigenFaces reshape(EigenVectors(:,j)+mean_face,m,n)];
    end;
end
EigenFaces=uint8(EigenFaces);
figure;
imshow(EigenFaces);

```

If you did everything correctly, you will see an image containing 29 eigenfaces, the first five of which are shown in Fig. 16. Observe that the mean face has been added back to the eigenfaces.

Variables: `EigenFaces`

6. The matrix $\mathbf{P}\mathbf{P}^T$ is symmetric. It follows from the theorems of linear algebra that the eigenvectors corresponding to the distinct eigenvalues are orthogonal. Write code to verify the last statement. You will need to create the matrix

`Products=EigenVectors'*EigenVectors`

and observe that it is a diagonal matrix. This matrix contains all possible dot products of the eigenvectors of the matrix $\mathbf{P}\mathbf{P}^T$. The fact that the eigenvectors are orthogonal will be used in the following sections to find orthogonal projections.

Variables: `Products`

Q2: Why is the matrix $\mathbf{P}\mathbf{P}^T$ symmetric?

7. In this project we will use the obtained eigenfaces to perform several operations. Our first task will be to “unmask” an altered face (sunglasses have been digitally added to the image). The main idea behind this code is in computing the projection of the vector of data (obtained from the altered image) onto the subspace spanned by the eigenvectors. This



Figure 16: The first five eigenfaces

projection is then displayed on the screen. Now run the code cell titled `%% Recognition of an altered image (sunglasses)`:

```
%% Recognition of an altered image (sunglasses)
image_read=imread(['database\person30altered1.pgm']);
U=reshape(image_read,m*n,1);
NormsEigenVectors=diag(Products);
W=(EigenVectors'*(double(U)-mean_face));
W=W./NormsEigenVectors;
U_approx=EigenVectors*W+mean_face;
image_approx=uint8(reshape(U_approx,m,n));
figure;
imshow([image_read,image_approx])
```

In this code, we start by uploading the image `person30altered1.pgm` into Matlab. The image is then reshaped as a long vector similarly to the way it was done before. Then we compute the projection of the new vector with the subtracted vector `mean_face` onto the space spanned by the set of the eigenvectors and add the vector `mean_face` back. Finally, the reconstructed image is displayed. Observe that we get a very good approximation of the initial image without sunglasses.

Variables: `image_read`, `U`, `NormsEigenVectors`, `W`, `U_approx`

8. Modify the code above to unmask the image `person30altered2.pgm`.
9. Now let us compute the approximation of the face which is not in the database. To do this, modify the code above again to approximate the image `person31.pgm`. Observe, that the results of the last two approximations are somewhat worse than the very first one. This can be explained by the fact that there is an insufficient number of images in our database which do not span a wide enough variety of faces.

Project 12: Matrix eigenvalues and the Google's PageRank algorithm

Goals: To apply matrix eigenvalues and eigenvectors to ranking of webpages in the World Wide Web.

To get started:

- Download the file `lab12.m` and put it in your working directory¹⁵.
- Download the file `AdjMatrix.mat` which contains the adjacency matrix of a so-called “wiki-vote” network with 8297 nodes¹⁶ [9].

Matlab commands used: `load`, `size`, `numel`, `nnz`, `for... end`, `gplot`

What you have to submit: The file `lab12.m` which you will modify during the lab session.

INTRODUCTION

According to social polls, the majority of users only look at the first few results of the online search and very few users look past the first page of results. Hence, it is crucially important to rank the pages in the “right” order so that the most respectable and relevant results will come first. The simplest way to determine the rank of a webpage in a network is to look at how many times it has been referred to by other webpages. This simple ranking method leaves a lot to be desired. In particular, it can be easily manipulated by referring to a certain webpage from a lot of “junk” webpages. The quality of the webpages referring to the page we are trying to rank should matter too. This is the main idea behind the Google PageRank algorithm.

The Google PageRank algorithm is the oldest algorithm used by Google to rank the web pages which are preranked offline. The PageRank scores of the webpages are recomputed each time Google crawls the web. Let us look at the theory behind the algorithm. As it turns out it, it is based on the theorems of linear algebra!

The main assumption of the algorithm is that if you are located on any webpage then with equal probability you can follow any of the hyperlinks from that page to another page. This allows to represent a webpage network as a directed graph with the webpages being the nodes, and the edges being the hyperlinks between the webpages. The adjacency matrix of such a network is built in the following way: the (i, j) th element of this matrix is equal to 1 if there is a hyperlink from the webpage i to the webpage j and is equal to 0 otherwise. Then the row sums of this matrix will represent numbers of hyperlinks from each webpage and the column sums will represent numbers of times each webpage has been referred to by other webpages.

Even further, we can generate a matrix of probabilities $\mathbf{S}$ such that the (i, j) th element of this matrix is equal to the probability of traveling from i th webpage to j th webpage in the network. This probability is equal to zero if there is no hyperlink from i th page to j th page and is equal to $1/N_i$ if there is a hyperlink from i th page to j th page, where N_i is the total number of hyperlinks from i th page. For instance, consider a sample network of only four webpages shown on the Fig. 17. The matrix $\mathbf{S}$ for this network can be written as:

¹⁵The printout of the file `lab12.m` is given in the appendices of this book.

¹⁶The original network data is available here: <https://snap.stanford.edu/data/>.

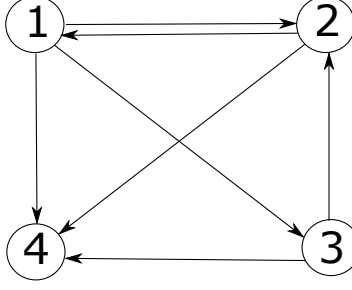


Figure 17: Sample network of four webpages

$$\mathbf{S} = \begin{bmatrix} 0 & 1/3 & 1/3 & 1/3 \\ 1/2 & 0 & 0 & 1/2 \\ 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (1)$$

There are several issues which make working with the matrix $\mathbf{S}$ inconvenient. First of all, there are webpages that do not have any hyperlinks - the so-called “dangling nodes” (such as the node 4 in Fig. 17). These nodes will correspond to zero rows of the matrix $\mathbf{S}$. Moreover, the webpages in the network may not be connected to each other and the graph of the network may consist of several disconnected components. These possibilities lead to undesirable properties of the matrix $\mathbf{S}$ which make computations with it complicated and not even always possible.

The problem of the dangling nodes can be solved by assigning all elements of the matrix $\mathbf{S}$ in the rows corresponding to the dangling nodes equal probabilities $1/N$, where N is the number of the nodes in the network. This can be understood in the following way: if we are at the dangling node we can with equal probability jump to any other page in the network. To solve the potential disconnectedness problem, we assume that a user can follow hyperlinks on any page with a probability $1 - \alpha$ and can jump (or “teleport”) to any other page in the network with a probability α . The number α is called a damping factor. The value of $\alpha = 0.15$ is usually taken in practical applications. The “teleport” surfing of the network can be interpreted as a user manually typing the webpage address in the browser or using a saved hyperlink from their bookmarks to move from one page onto another. The introduction of the damping factor allows us to obtain the Google matrix $\mathbf{G}$ in the form:

$$\mathbf{G} = (1 - \alpha)\mathbf{S} + \alpha\mathbf{E},$$

where $\mathbf{E}$ is a matrix with all the elements equal to $1/N$, where N is a number of webpages in the network.

The matrix $\mathbf{G}$ has nice properties. In particular, it has only positive entries and all of its rows sum up to 1. In mathematical language, this matrix is *stochastic* and *irreducible* (you can look up the precise definitions of these terms if you are interested). The matrix $\mathbf{G}$ satisfies the following **Perron-Frobenius theorem**:

Theorem 1 (Perron-Frobenius) *Every square matrix with positive entries has a unique unit eigenvector with all positive entries. The eigenvalue corresponding to this eigenvector is real and positive. Moreover, this eigenvalue is simple and is the largest in absolute value among all the eigenvalues of this matrix.*

Let us apply this theorem to the matrix $\mathbf{G}$. First of all, observe that the row sums of the

matrix $\mathbf{G}$ are equal to 1. Consider the vector $\mathbf{v}_1 = (1, 1, \dots, 1)^T/N$. It is easy to see that

$$\mathbf{G}\mathbf{v}_1 = \mathbf{v}_1.$$

But then it follows that $\mathbf{v}_1$ is the unique eigenvector with all positive components, and, therefore, by the Perron-Frobenius theorem, $\lambda_1 = 1$ is the largest eigenvalue!

We are interested in the left eigenvector for the eigenvalue $\lambda_1 = 1$:

$$\mathbf{u}_1^T \mathbf{G} = \mathbf{u}_1^T.$$

Again, by the Perron-Frobenius theorem, the vector $\mathbf{u}_1$ is the unique unit eigenvector with all positive components corresponding to the largest in absolute value eigenvalue $\lambda_1 = 1$. We will use the components of this vector for the ranking of webpages in the network.

Let us look at the justification behind this algorithm. We have already established that the vector $\mathbf{u}_1$ exists. Consider the following iterative process. Assume that at the beginning a user can be on any webpage in the network with equal probability:

$$\mathbf{w}_0 = (1/N, 1/N, \dots, 1/N).$$

After 1 step (one move from one webpage to another using hyperlinks or teleporting), the probability vector of being on the i th webpage is determined by the i th component of the vector

$$\mathbf{w}_1 = \mathbf{w}_0 \mathbf{G}.$$

After two moves the vector of probabilities becomes

$$\mathbf{w}_2 = \mathbf{w}_1 \mathbf{G} = \mathbf{w}_0 \mathbf{G}^2,$$

and so on.

We hope that after a large number of steps n , the vector $\mathbf{w}_n = \mathbf{w}_0 \mathbf{G}^n$ starts approaching some kind of limit vector $\mathbf{w}_*$, $\mathbf{w}_n \rightarrow \mathbf{w}_*$. It turns out that due to the properties of the matrix $\mathbf{G}$ this limit vector $\mathbf{w}_*$ indeed exists and it is exactly the eigenvector corresponding to the largest eigenvalue, namely, $\lambda_1 = 1$. Moreover, numerical computation of matrix eigenvalues is actually based on taking the powers of the matrix (it is called the Power method) and not on solving the characteristic equation!

Let us assume that the vector $\mathbf{w}_*$ is a non-negative vector whose entries sum to 1. Then the components of this vector represent the probabilities of being on each webpage in the network after a very large number of moves along the hyperlinks. Thus, it is perfectly reasonable to take these probabilities as ranking of the webpages in the network.

TASKS

1. Open the file `lab12.m`. In the code cell titled `%%Load the network data` load the data from the file `AdjMatrix.mat` into Matlab by using the `load` command. Save the resulting matrix as `AdjMatrix`. Observe that the adjacency matrices of real networks are likely to be very large (may contain millions of nodes or more) and sparse. Check the sparsity of the matrix `AdjMatrix` using the functions `numel` and `nnz`. Denote the ratio of non-zero elements as `nnzAdjMatrix`. If you did everything correctly you should obtain that only 0.15% of the elements of the matrix `AdjMatrix` are non-zero.

Variables: `AdjMatrix`, `nnzAdjMatrix`

2. Check the dimensions of the matrix `AdjMatrix` using the `size` function. Save the dimensions as new variables `m` and `n`.

Variables: `m`, `n`

3. Observe that while the network described by the matrix `AdjMatrix` is not large at all from the viewpoint of practical applications, computations with this matrix may still take a noticeable amount of time. To save time, we will cut a subset out of this network and use it to illustrate the Google PageRank algorithm. Introduce a new variable `NumNetwork` and set its value to 500. Then cut a submatrix `AdjMatrixSmall` out of the matrix `AdjMatrix` and plot the graph represented by the matrix `AdjMatrixSmall` by running the following code cell:

```
%% Display a small amount of network
NumNetwork=500;
AdjMatrixSmall=AdjMatrix(1:NumNetwork,1:NumNetwork);
for j=1:NumNetwork
    coordinates(j,1)=NumNetwork*rand;
    coordinates(j,2)=NumNetwork*rand;
end;
gplot(AdjMatrixSmall,coordinates,'k-*');
```

This will plot the subgraph of the first 500 nodes in the network with random locations of the nodes. Notice the use of the function `gplot` to produce this graph. Observe that Matlab has special functions `graph` and `digraph` for working with graphs, but those functions are a part of the special package “Graph and Network Algorithms” which may not be immediately available. Simpler methods, as shown above, will be sufficient for our purposes.

Variables: `AdjMatrixSmall`, `coordinates`, `NumNetwork`

4. Set the parameter $\alpha = 0.15$. Introduce the vector $w_0 = (1, 1, \dots, 1) / \text{NumNetwork}$, and compute the consequent vectors $w_1 = w_0 G$, $w_2 = w_1 G$, $w_3 = w_2 G$, $w_5 = w_0 G^5$, $w_{10} = w_0 G^{10}$. Compute the difference $\delta w = w_{10} - w_5$. Observe that the sequence w_n converges to a certain limit vector w_* very fast.

Variables: `w0`, `w1`, `w2`, `w3`, `w5`, `w10`, `deltaw`

5. Compute the eigenvalues and the left and the right eigenvectors of the matrix G using the function `eig`. Observe that the right eigenvector corresponding to the eigenvalue $\lambda_1 = 1$ is proportional to the vector $v_1 = (1, 1, \dots, 1)$. To compute the left eigenvectors, use the function `eig` on the matrix G' . Select the left eigenvector corresponding to the eigenvalue $\lambda_1 = 1$ and denote it as `u1`.

Variables: `u1`

6. Observe that by default the vector `u1` is not scaled to have all positive components (even though all the components of the vector `u1` will have the same sign). Normalize this vector by using the code:

```
u1=abs(u1)/norm(u1,1);
```

This will create a vector with all positive components whose entries sum to 1 (called a probability vector).

7. Use the function `max` to select the maximal element and its index in the array.

Variables: `MaxRank`, `PageMaxRank`

8. Find out whether the highest ranking webpage is the same as the page with the most hyperlinks pointed to it. To do so, create the vector of column sums of the matrix **G** and save it as **MostLinks**. Use the function **max** again to select the element with the maximal number of links.

Variables: **MostLinks**, **MaxLinks**, **PageMaxLinks**

9. Compare if **MaxRank** and **MaxLinks** are the same.

Q1: What is the number of hyperlinks pointing to the webpage **MaxRank**?

(look at the vector **MostLinks** to find out).

Project 13: Social networks, clustering, and eigenvalue problems

Goals: To apply linear algebra techniques to data clustering.

To get started:

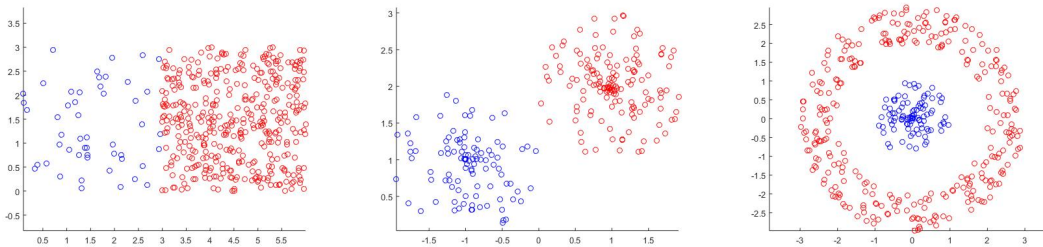
- Download the file `lab13.m` and put it in your working directory¹⁷.
- Download the file `social.mat` which contains the adjacency matrix of a simulated social network¹⁸.

Matlab commands used: `sum`, `diag`, `eig`, `for...end`, `if... then... end`, `load`, `clear`, `spy`, `eye`

What you have to submit: The file `lab13.m` which you will modify during the lab session.

INTRODUCTION

Many practical applications deal with large amounts of data. One of the standard problems in data science is to group this data in clusters containing similar items. In many cases, clustering can be easily accomplished by the human brain (take a look at the attached pictures).



In general, creating an algorithm which allows us to separate the data into clusters is a difficult task. The data in many cases is multidimensional and cannot be visualized easily. Even deciding how many clusters to select is not necessary an easy task. There are currently several widely used algorithms which allow for the separation of the data points by similarity. All these algorithms have their own advantages and disadvantages.

In this project, we will use the spectral clustering algorithm based on the Fiedler's vector. We will look at the graph representing an individual's profile on a social network. The profile has a total of 351 "friends". Some of these friends know each other and are friends with each other as well. This friendship network can be represented as a graph with nodes being friends and edges showing the friendship connection between the individuals. The goal of clustering is to separate this graph into components which are tightly connected within themselves and have fewer connections to the outside (the so-called communities). The method explored here is based on the theorem by M. Fiedler, who has shown that for a connected graph (meaning that there is a path between any two vertices) the eigenvector corresponding to the second smallest eigenvalue of the Laplacian of the graph allows the graph to be split into two maximally intraconnected components.

¹⁷The printout of the file `lab13.m` can be found in the appendices of this book.

¹⁸This data has been artificially created.

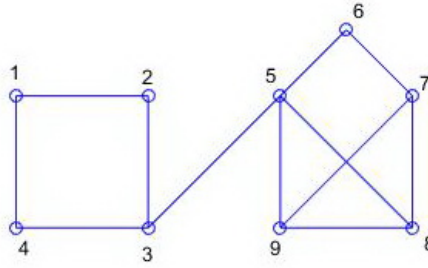


Figure 18: An example of a simple graph

TASKS

1. We will start by considering an example of a simple graph: Create a new code cell with the title `%% Small graph`. Type in the adjacency matrix of this graph into Matlab and save it in a variable `AdjMatrix`.

Variables: `AdjMatrix`

2. Find the row sums of the matrix `AdjMatrix` (use the `sum` function) and save it as `RowSums`.

Variables: `RowSums`

3. Compute the Laplacian of the graph using the following command:

```
LaplaceGraph=diag(RowSums)-AdjMatrix;
```

Observe that the `diag(RowSums)` command creates a diagonal matrix with the vector `RowSums` on the main diagonal and zeros everywhere else. Observe also that the matrix `LaplaceGraph` is a symmetric positive semi-definite singular matrix. Recall that a matrix $\mathbf{A}$ is positive semi-definite if $\mathbf{xAx}^T \geq 0$ for all column vectors $\mathbf{x}$. To check that it is singular, multiply the matrix `LaplaceGraph` on the right by the column vector $[1, 1, 1, 1, 1, 1, 1, 1, 1]^T$.

Variables: `LaplaceGraph`

4. Find the eigenvalues and eigenvectors of the matrix `LaplaceGraph` using the `eig` function:

```
[V,D]=eig(LaplaceGraph);
```

Here the matrix `V` contains all the right eigenvectors of the matrix `LaplaceGraph` and the matrix `D` is a diagonal matrix with eigenvalues of the matrix `LaplaceGraph` on the main diagonal. Since the Laplacian is a symmetric graph, its eigenvalues are real. To simplify our task, let us sort the eigenvalues from smallest to largest. We will need to sort the eigenvectors along with them. Here is how to do that.

```
% sort eigenvalues and eigenvectors
[d,ind] = sort(diag(D));
D = D(ind,ind);
V = V(:,ind);
```

Variables: V, D

5. View the elements of the matrices **V** and **D** using the “Workspace” window of the Matlab environment. Observe that one of the eigenvalues of the matrix **LaplaceGraph** is zero (this is due to the fact that the matrix **LaplaceGraph** is singular). The eigenvector for this eigenvalue is proportional to the vector $[1, 1, 1, 1, 1, 1, 1, 1, 1]^T$. All other eigenvalues of the matrix **LaplaceGraph** are positive. This follows from the fact that the matrix **LaplaceGraph** is positive semi-definite and the graph corresponding to this Laplacian is connected. Find the smallest positive eigenvalue of the matrix **LaplaceGraph** (since we sorted the eigenvalues, it is the second diagonal element of the matrix **D**).
6. Identify the eigenvector corresponding to the second smallest eigenvalue:

```
V2=V(:,2);
```

In the next step, we will use the entries of **V2** to group the elements. To make sure we do this consistently, let us make sure that **V2** is the unit eigenvector with positive first entry:

```
% make sure the first entry is positive
if V2(1) < 0
    V2 = -V2;
end
```

Variables: V2

7. Separate the elements of the eigenvector **V2** onto positive and negative:

```
pos=[];
neg=[];
for j=1:9
    if V2(j)>0
        pos=[pos,j];
    else
        neg=[neg,j];
    end;
end;
```

If everything is done correctly, the arrays **pos** and **neg** will contain correspondingly 1, 2, 3, 4 and 5, 6, 7, 8, 9 which would be an intuitive clustering choice.

Variables: pos, neg

8. Now let us apply this technique to a larger graph when the results cannot be visualized so easily. To do this, load the matrix **Social** which is an adjacency matrix of a network of 351 Facebook friends. The (i, j) th element is equal to 1 if the i th and j th friends are also friends with each other and is equal to 0 otherwise:

```
%% Load the data
clear;
load('social.mat','Social');
spy(Social);
```

The command **spy(Social)** will plot the sparsity pattern of the adjacency matrix **Social** on a figure, putting blue dots for non-zero elements of the matrix (you should see something similar to the first image in Fig. 19). Observe that it is difficult to distinguish any kind of underlying pattern in this matrix. Our goal will be to cluster this set of data into two

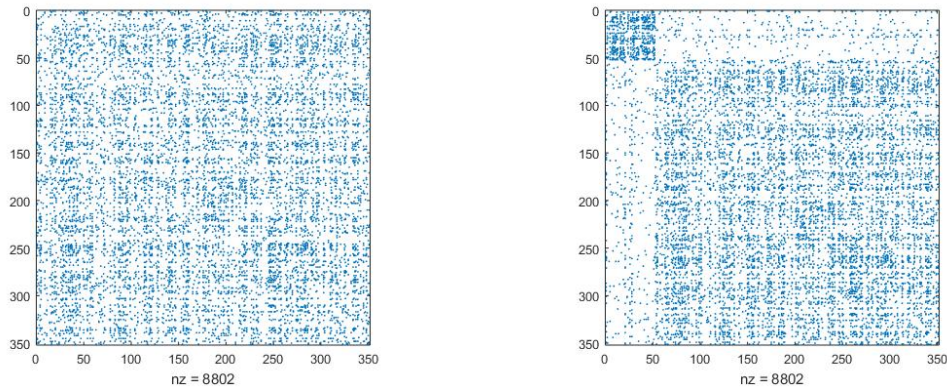


Figure 19: Adjacency matrices of the unsorted and sorted data.

maximally intraconnected groups.

Variables: **Social**

9. To do this, repeat the procedure we performed on a smaller matrix. Define the matrices **DiagSocial** and **LaplaceSocial** which are correspondingly the vector of row sums of the matrix **Social** and the Laplacian of this matrix.

Variables: **DiagSocial, LaplaceSocial**

10. Compute the eigenvalues and the eigenvectors:

```
[V,D]=eig(LaplaceSocial);
```

Variables: **V,D**

11. As before, identify the second smallest eigenvalue and the corresponding eigenvector **V2** and sort the vertices of the graph by the positive and negative components of the vector **V2**. Make the necessary adjustment to ensure that **V2(1)>0**.

Variables: **V2**

12. Permute the adjacency matrix **Social** using the permutation generated by the lists **pos**, **neg**:

```
order=[pos,neg];
[m,n]=size(Social);
iden=eye(m,m);
for j=1:351;
    for k=1:351;
        P(j,k)=iden(order(j),k);
    end;
end;
SocialOrdered=P*Social*P'
```

Variables: **order, P, SocialOrdered**

Q1: What does the command **P*Social*P'** do to the rows and columns of the matrix **Social**?

13. Plot the matrix `SocialOrdered` using the function `spy` again. If you did everything correctly, you will see the matrix separating into two denser connected subsets similarly to what is pictured in the second image in Fig. 19.
14. What if we want more clusters? There are two ways to proceed. First, we can explore the eigenvector `V3` corresponding to the third smallest eigenvalue. We will group the nodes in the four clusters: the ones which have positive components in both eigenvectors `V2`, `V3` (`++` group), the ones which have a positive component in `V2` and a negative component in `V3` (`+-`), the ones which have a negative component in `V2` and a positive component in `V3` (`-+`), and, finally, the ones with negative components in both vectors (`--`). This generates 4 distinct groups and can be accomplished by the code in the following code cell:

```
%% Cluster in 4 groups
V3=V(:,3);
if V3(1) < 0
    V3 = -V3;
end
pp=[];
pn=[];
np=[];
nn=[];
for j=1:m
    if (V2(j)>0)
        if (V3(j)>0)
            pp=[pp,j];
        else
            pn=[pn,j];
        end;
    else
        if (V3(j)>0)
            np=[np,j];
        else
            nn=[nn,j];
        end;
    end;
end;
order=[pp,pn,np,nn];
iden=eye(m,m);
for j=1:351;
    for k=1:351;
        P(j,k)=iden(order(j),k);
    end;
end;
SocialOrdered=P*Social*P'
figure;
spy(SocialOrdered)
```

15. An alternative way to obtain more clusters is to use the Fiedler vector procedure iteratively, meaning that we will apply it again to the clusters obtained in the previous step. This can be accomplished by running the following code:

```
%% Second order of Fiedler
SocialPos=Social(pos,pos);
SocialNeg=Social(neg,neg);
rowsumpos=sum(SocialPos,2);
```

```

DiagSocialPos=diag(rowsumpos);
LaplaceSocialPos=DiagSocialPos-SocialPos;
[VPos,DPos]=eig(LaplaceSocialPos);
V2Pos=VPos(:,2);
[mpos,npos]=size(V2Pos);
posp=[];
posn=[];
for j=1:mpos
    if (V2Pos(j)>0)
        posp=[posp,pos(j)];
    else
        posn=[posn,pos(j)];
    end;
end;
rowsumneg=sum(SocialNeg,2);
DiagSocialNeg=diag(rowsumneg);
LaplaceSocialNeg=DiagSocialNeg-SocialNeg;
[VNeg,DNeg]=eig(LaplaceSocialNeg);
V2Neg=VNeg(:,2);
[mneg,nneg]=size(V2Neg);
negp=[];
negn=[];
for j=1:mneg
    if (V2Neg(j)>0)
        negp=[negp,neg(j)];
    else
        negn=[negn,neg(j)];
    end;
end;
ordergen=[posp,posn,negp,negn];
iden=eye(m,m);
for j=1:351;
    for k=1:351;
        P(j,k)=iden(ordergen(j),k);
    end;
end;
SocialOrderedGen=P*Social*P'
figure;
spy(SocialOrderedGen)

```

Observe that the densely connected components obtained by the last two methods might be different. In the context of a social network, the dense clusters may represent groups of people who have common connections in real life. For instance, your family, your friends from high school, your colleagues, etc.

Appendices

Printout of the file lab09.m

```
%%
clear;
Vertices=[1 1 1;...
          -1 1 1;...
           1 -1 1;...
           1 1 -1;...
          -1 -1 1;...
          -1 1 -1;...
           1 -1 -1;...
          -1 -1 -1];

Edges=zeros(8,8);
Edges(1,2)=1;
Edges(1,3)=1;
Edges(1,4)=1;
Edges(2,5)=1;
Edges(2,6)=1;
Edges(3,5)=1;
Edges(3,7)=1;
Edges(4,6)=1;
Edges(4,7)=1;
Edges(5,8)=1;
Edges(6,8)=1;
Edges(7,8)=1;

Edges=Edges+Edges';

theta1=pi/3;
theta2=pi/4;
theta3=pi/6;

rotmat=rotation(theta1,theta2,theta3);

VertRot=Vertices*rotmat;

figure;
axis equal;
hold on;

for j=1:8
    for k=j+1:8
        if (Edges(j,k)==1)
            line([VertRot(j,1),VertRot(k,1)], [VertRot(j,2),VertRot(k,2)]);
        end;
    end;
end;

hold off;

%% Buckyball
```

```

[Edges,Vertices] = bucky;
figure;
axis equal;
hold on;

theta1=pi/3;
theta2=pi/4;
theta3=pi/6;

rotmat=rotation(theta1,theta2,theta3);

VertRot=Vertices*rotmat;

for j=1:60
    for k=j+1:60
        if (Edges(j,k)==1)
            line([VertRot(j,1),VertRot(k,1)],[VertRot(j,2),VertRot(k,2)]);
        end;
    end;
end;

hold off;

figure;
axis equal;
hold on;

for j=1:60
    for k=j+1:60
        if (Edges(j,k)==1)
            line([Vertices(j,1),Vertices(k,1)],[Vertices(j,2),Vertices(k,2)],...
                [Vertices(j,3),Vertices(k,3)]);
        end;
    end;
end;

hold off;

%% 3D fawn
clear;

load('v.mat','v');
load('f.mat','f');

[mVert,nVert]=size(v);

[mFace,nFace]=size(f);

figure;
axis equal;
hold on;
for j=1:mFace
    line([v(f(j,1),1),v(f(j,2),1)],[v(f(j,1),2),v(f(j,2),2)],[v(f(j,1),3),v(f(j,2),3)]);

```

```

        line([v(f(j,1),1),v(f(j,3),1)], [v(f(j,1),2),v(f(j,3),2)], [v(f(j,1),3),v(f(j,3),3)]);
        line([v(f(j,2),1),v(f(j,3),1)], [v(f(j,2),2),v(f(j,3),2)], [v(f(j,2),3),v(f(j,3),3)]);
    end;
    hold off;

    figure;
    axis equal;
    hold on;

    theta1=pi/3;
    theta2=pi/4;
    theta3=pi/2;

    rotmat=rotation(theta1,theta2,theta3);

    VertRot=v*rotmat;

    for j=1:mFace
        line([VertRot(f(j,1),1),VertRot(f(j,2),1)], [VertRot(f(j,1),2),VertRot(f(j,2),2)]);
        line([VertRot(f(j,1),1),VertRot(f(j,3),1)], [VertRot(f(j,1),2),VertRot(f(j,3),2)]);
        line([VertRot(f(j,2),1),VertRot(f(j,3),1)], [VertRot(f(j,2),2),VertRot(f(j,3),2)]);
    end;

```

Printout of the file lab10.m

```
% The Chaos Game

%% Sierpinski triangle

%% Sierpinski triangle with a change of ratio

%% Sierpinski triangle with rotation

%% Square

%% Square with different ratio

%% Square with vertex preference (new vertex cannot be the same as previous)
clear;

T=[1/2 0; 0 1/2];

t=linspace(0,2*pi,5);
t(5)=[];
v=[cos(t); sin(t)];

x(:,1)=[rand-0.5; rand-0.5];

plot(v(1,:),v(2,:), 'k*', x(1,1), x(2,1), 'b. ');
axis equal;
hold on;

Num=5000;

k1=0;
for j=1:Num
    k=randi(3);
    if (k>=k1)
        k=k+1;
    end;
    x(:,j+1)=T*(x(:,j)-v(:,k))+v(:,k);
    k1=k;
end;

plot(x(1,:),x(2,:), 'b. ');
hold off;

%% Square with vertex preference (new vertex cannot be opposite of the previous)
clear;

T=[1/2 0; 0 1/2];

t=linspace(0,2*pi,5);
t(5)=[];
v=[cos(t); sin(t)];

x(:,1)=[rand-0.5; rand-0.5];
```

```

plot(v(1,:),v(2,:), 'k*', x(1,1), x(2,1), 'b. ');
axis equal;
hold on;

Num=5000;

w=[real(v(1));imag(v(1))];
k1=0;

for j=1:Num
    k=randi(4)
    if (k~=k1+2)&&(k1~=k+2)
        x(:,j+1)=T*(x(:,j)-v(:,k))+v(:,k);
        k1=k;
    else
        x(:,j+1)=x(:,j);
    end;
end;

plot(x(1,:),x(2,:), 'b. ');
hold off;

%% Barnsley fern
clear;

T1=[0.85,0.04;-0.04,0.85];
T2=[-0.15,0.28;0.26,0.24];
T3=[0.2,-0.26;0.23,0.22];
T4=[0,0;0,0.16];
Q1=[0;1.64];
Q2=[-0.028;1.05];
Q3=[0;1.6];
Q4=[0,0];
P1=0.85;
P2=0.07;
P3=0.07;
P4=0.01;

Num=15000;
x(:,1)=rand(2,1);
plot(x(1,:),x(2,:), 'b. ')
axis equal;
hold on;
for j=1:Num
    r=rand;
    if r<=P1
        x(:,j+1)=T1*x(:,j)+Q1;
    elseif r<=P1+P2
        x(:,j+1)=T2*x(:,j)+Q2;
    elseif r<=P1+P2+P3
        x(:,j+1)=T3*x(:,j)+Q3;
    else
        x(:,j+1)=T4*x(:,j);
    end;
end;

```



```

        end
    end

    plot(x(1,:),x(2,:), 'b. ');
    hold off;

    %% Hexagon
    clear;

    T=[1/3 0; 0 1/3];

    t=linspace(0,2*pi,7);
    t(7)=[];
    v=[cos(t); sin(t)];

    x(:,1)=[rand-0.5; rand-0.5];

    plot(v(1,:),v(2,:), 'k*', x(1,1), x(2,1), 'b. ');
    axis equal;
    hold on;

    Num=10000;

    for j=1:Num
        k=randi(6);
        x(:,j+1)=T*(x(:,j)-v(:,k))+v(:,k);
    end;
    plot(x(1,:),x(2,:), 'b. ');
    hold off;

    %% Pentagon with a skipped vertex

    clear;
    T=[1/2.5 0; 0 1/2.5];

    t=linspace(0,2*pi,6);
    t(6)=[];
    v=[cos(t); sin(t)];

    x(:,1)=[rand-0.5; rand-0.5];

    plot(v(1,:),v(2,:), 'k*', x(1,1), x(2,1), 'b. ');
    axis equal;
    hold on;

    Num=10000;

    k1=0;

    for j=1:Num
        k=randi(5);
        if k~=k1
            x(:,j+1)=T*(x(:,j)-v(:,k))+v(:,k);

```

```
        k1=k;  
    else  
        x(:,j+1)=x(:,j);  
    end;  
end;  
plot(x(1,:),x(2,:), 'b. ');  
hold off;
```

Printout of the file lab11.m

```
% Simple face recognition algorithm
%% Input database files into the Matlab
clear;
Database_Size=30;

%Reading images from the database. The image files should be located in the
%subfolder "database"
for j=1:Database_Size
    image_read=imread(['database\person' num2str(j) '.pgm']);
    [m,n]=size(image_read);
    P(:,j)=reshape(image_read,m*n,1);
end;

%% Computing and displaying the mean face

%% Subtract the mean face

%% Compute the covariance matrix of the set and its eigenvalues and eigenvectors

%% Display the set of eigenvaces
for j=2:Database_Size;
    if j==2
        EigenFaces=reshape(EigenVectors(:,j)+mean_face,m,n);
    else
        EigenFaces=[EigenFaces reshape(EigenVectors(:,j)+mean_face,m,n)];
    end;
end

EigenFaces=uint8(EigenFaces);

figure;
imshow(EigenFaces);

%% Orthogonality and symmetry

%% Recognition of an altered image (sunglasses)
image_read=imread(['database\person30altered1.pgm']);
U=reshape(image_read,m*n,1);

NormsEigenVectors=diag(Products);
W=(EigenVectors'*(double(U)-mean_face));
W=W./NormsEigenVectors;

U_approx=EigenVectors*W+mean_face;

image_approx=uint8(reshape(U_approx,m,n));
figure;
imshow([image_read,image_approx])

%% Recognition of an altered image (lower part of the face)

%% Recognition of a person not in the database
```

Printout of the file lab12.m

```
% Google PageRank algorithm on the example of random network

%% Load the network data
clear;

%% Display a small amount of network
NumNetwork=500;
AdjMatrixSmall=AdjMatrix(1:NumNetwork,1:NumNetwork);

for j=1:NumNetwork
    coordinates(j,1)=NumNetwork*rand;
    coordinates(j,2)=NumNetwork*rand;
end;

gplot(AdjMatrixSmall,coordinates,'k-*');

%% Check the amount of links originating from each webpage
NumLinks=sum(AdjMatrixSmall,2);

%% Create a matrix of probabilities (Google matrix)
% Element (i,j) of the matrix shows the probability of moving from i-th
% page of the network to jth page. It is assumed that the user can follow
% any link on the page with a total probability of 85% (all hyperlinks are
% equal), and jump (teleport) to any other page in the network with a total
% probability of 15% (again, all pages are equal).

alpha=0.15;
GoogleMatrix=zeros(NumNetwork,NumNetwork);
for i=1:NumNetwork
    if NumLinks(i)~=0
        GoogleMatrix(i,:)=AdjMatrixSmall(i,:)./NumLinks(i);
    else
        GoogleMatrix(i,:)=1./NumNetwork;
    end;
end;

GoogleMatrix=(1-alpha)*GoogleMatrix+alpha*ones(NumNetwork,NumNetwork)./NumNetwork;

%% Check that all the rows in the GoogleMatrix matrix sum to 1
SumGoogleMatrix=sum(GoogleMatrix,2);

%% Finding an eigenvector corresponding to 1 (why is there such an eigenvector)?
w0=ones(1,NumNetwork)./sqrt(NumNetwork);

w1=w0*GoogleMatrix;
w2=w1*GoogleMatrix;
w3=w2*GoogleMatrix;

w100=w0*(GoogleMatrix)^100;
w90=w0*(GoogleMatrix)^90;
```

```

% Check the difference between v30 and v20. Observe that it is pretty
% small
deltaw=w100-w90;

%% Compute the eigenvalues and the right eigenvectors
[VectRight,LamRight]=eig(GoogleMatrix);
% Explain the result
LamRight=diag(LamRight);

%% Compute the eigenvalues and the left eigenvectors
[VectLeft,LamLeft]=eig(GoogleMatrix');
LamLeft=diag(LamLeft);

%% Separate the eigenvector corresponding to the eigenvalue 1 and scale it
u1=VectLeft(:,1);

u1=abs(u1)/norm(u1);

%% Select the maximum element and the corresponding element.
%Which page is the most important in the network?

[MaxRank,PageMaxRank]=max(u1);

%% Check if it's the most popular (most linked to page):

MostLinks=sum(AdjMatrixSmall);
[MaxLinks,PageMaxLinks]=max(MostLinks);

```

Printout of the file lab13.m

```
% Facebook and graph partitioning

%% Simple graph

%% Load the data
clear;
load('social.mat','Social');
spy(Social);

%% Laplacian

%% Spectral decomposition

%% Clusteer in two groups

%% Cluster in 4 groups

Diagonal(indsmall)=large+1000;
[small,indsmall]=min(Diagonal);

V3=V(:,indsmall);

pp=[];
pn=[];
np=[];
nn=[];

for j=1:m
    if (V2(j)>0)
        if (V3(j)>0)
            pp=[pp,j];
        else
            pn=[pn,j];
        end;
    else
        if (V3(j)>0)
            np=[np,j];
        else
            nn=[nn,j];
        end;
    end;
end;

order=[pp,pn,np,nn];

iden=eye(m,m);

for j=1:351;
    for k=1:351;
        P(j,k)=iden(order(j),k);
    end;
end;
```

```

SocialOrdered=P*Social*P'

figure;
spy(SocialOrdered)

%% Second order of Fiedler

SocialPos=Social(pos,pos);
SocialNeg=Social(neg,neg);

rowsumpos=sum(SocialPos,2);

DiagSocialPos=diag(rowsumpos);

LaplaceSocialPos=DiagSocialPos-SocialPos;

[VPos,DPos]=eig(LaplaceSocialPos);

V2Pos=VPos(:,2);

[mpos,npos]=size(V2Pos);

posp=[];
posn=[];

for j=1:mpos
    if (V2Pos(j)>0)
        posp=[posp,pos(j)];
    else
        posn=[posn,pos(j)];
    end;
end;

rowsumneg=sum(SocialNeg,2);

DiagSocialNeg=diag(rowsumneg);

LaplaceSocialNeg=DiagSocialNeg-SocialNeg;

[VNeg,DNeg]=eig(LaplaceSocialNeg);

V2Neg=VNeg(:,2);

[mneg,nneg]=size(V2Neg);

negp=[];
negn=[];

for j=1:mneg
    if (V2Neg(j)>0)
        negp=[negp,neg(j)];
    else
        negn=[negn,neg(j)];
    end;
end;

```

```

        end;
    end;

    ordergen=[posp,posn,negp,negn];

    iden=eye(m,m);

    for j=1:351;
        for k=1:351;
            P(j,k)=iden(ordergen(j),k);
        end;
    end;

    SocialOrderedGen=P*Social*P'
    figure;
    spy(SocialOrderedGen)

```


Printout of the file lab14.m

```

%% 2x2 matrix

clear;
t=linspace(0,2*pi,100);
X=[cos(t);sin(t)];
subplot(2,2,1);
hold on;
plot(X(1,:),X(2:,:), 'b');
quiver(0,0,1,0,0, 'r');
quiver(0,0,0,1,0, 'g');
axis equal
title('Unit circle')
hold off;

A = [ 2, 1; -1, 1 ];
[U,S,V] = svd(A);
U'*U
V'*V

VX=V'*X;
subplot(2,2,2)
hold on;
plot(VX(1,:),VX(2:,:), 'b');
quiver(0,0,V(1,1),V(1,2),0, 'r');
quiver(0,0,V(2,1),V(2,2),0, 'g');
axis equal
title('Multiplied by matrix V^T')
hold off;

SVX = S*VX;
subplot(2,2,3);
hold on;
plot(SVX(1,:),SVX(2:,:), 'b');
quiver(0,0,S(1,1)*V(1,1),S(2,2)*V(1,2),0, 'r');
quiver(0,0,S(1,1)*V(2,1),S(2,2)*V(2,2),0, 'g');
axis equal
title('Multiplied by matrix \Sigma V^T')
hold off;

AX = U*SVX;
subplot(2,2,4)
hold on;
plot(AX(1,:),AX(2:,:), 'b');
quiver(0,0,U(1,1)*S(1,1)*V(1,1)+U(1,2)*S(2,2)*V(1,2),U(2,1)*S(1,1)*V(1,1)+...
    U(2,2)*S(2,2)*V(1,2),0, 'r');
quiver(0,0,U(1,1)*S(1,1)*V(2,1)+U(1,2)*S(2,2)*V(2,2),U(2,1)*S(1,1)*V(2,1)+...
    U(2,2)*S(2,2)*V(2,2),0, 'g');
axis equal
title('Multiplied by matrix U\Sigma V^T=A')
hold off;

%% Modified SVD

```

```

U1(:,1)=U(:,1);
U1(:,2)=-U(:,2);
V1(:,1)=V(:,1);
V1(:,2)=-V(:,2);

U1*S*V1'-A

%% Check
A*V-U*S

%% Image compression

% Creates a two-dimensional array with the dimensions equal to the dimensions of
% the image
clear;
ImJPG=imread('einstein.jpg');
figure;
imshow(ImJPG);

[m,n]=size(ImJPG);

% Compute an SVD

[UIm,SIm,VIm]=svd(double(ImJPG));

% plot the singular values
figure;
plot(1:min(m,n),diag(SIm));

%% Create approximations to the image

% With 50, 100, and 150 singular values
for k=50:50:150
    ImJPG_comp=uint8(UIm(:,1:k)*SIm(1:k,1:k)*(VIm(:,1:k))');
    figure;
    imshow(ImJPG_comp)
    % compression percentage
    pct = 1 - (numel(UIm(:,1:k))+numel(VIm(:,1:k)*SIm(1:k,1:k)))/numel(ImJPG);
    fprintf('Compression percentage for %2.0f singular values: %8.3f\n',k, pct);
end;

%% Noise filtering
clear;
ImJPG=imread('checkers.pgm')
[m,n]=size(ImJPG);

% Add some noise to the image
ImJPG_Noisy=double(ImJPG)+50*(rand(m,n)-0.5*ones(m,n));
figure;
imshow(ImJPG);

figure;
imshow(uint8(ImJPG_Noisy));

```

```

[UIm,SIm,VIm]=svd(ImJPG_Noisy);

figure;
plot(1:min(m,n),diag(SIm),'ko');

for k=10:20:50
    ImJPG_comp=uint8(UIm(:,1:k)*SIm(1:k,1:k)*(VIm(:,1:k))');
    figure;
    imshow(ImJPG_comp)
    % compression percentage
    pct = 1 - (numel(UIm(:,1:k))+numel(VIm(:,1:k)*SIm(1:k,1:k)))/numel(ImJPG);
    fprintf('Compression percentage for %2.0f singular values: %8.3f\n',k, pct);
end;

```

References

- [1] S. Attaway. *Matlab: A practical Introduction to Programming and Problem solving*. Butterworth-Heinemann, 3rd edition, 2013.
- [2] K. Bryan, T. Leise. *The \$25,000,000,000 Eigenvector: The Linear Algebra behind Google*. SIAM Rev., **48(3)**, 569 - 581, 2006.
- [3] T. Chartier. *When Life is Linear: From Computer Graphics to Bracketology*. Mathematical Association of America, 2015.
- [4] W.N. Colley. *Colley's bias free college football ranking method: The Colley matrix explained*. www.Colleyrankings.com, 2002.
- [5] L. Elden. *Numerical linear algebra in data mining*. Acta Numerica, **15**, 327-384, 2006.
- [6] C. Hennig, M. Meila, F. Murthagh and R. Rocci (eds.). *Handbook of Cluster Analysis*. CRC Press, 2015.
- [7] J. Keener. *The Perron–Frobenius Theorem and the Ranking of Football Teams*. SIAM Rev., **35(1)**, 80-93, 1993.
- [8] A.N. Langville, C.D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, 2006.
- [9] J. Leskovec, R. Sasic. *SNAP: A General-Purpose Network Analysis and Graph-Mining Library*. ACM Transactions on Intelligent Systems and Technology (TIST), **8(1)**, 2016.
- [10] F.M. Harper, J.A. Konstan, *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems (TiiS), **5(4)**, 2015.